

DOM-like UI Trees

Definitions

This chapter is self-contained. We fix a few **primitive universes** and a small algebra of **finite maps**, then define elements, the tree, its state, the mutation algebra, and the event semantics, each in dependency order. Every per-element facet is a finite map, so the style cascade and reconciliation arise as instances of the map operations defined below rather than as new machinery.

Fix the following pairwise-disjoint **primitive universes** – the alphabet of the framework. A concrete instantiation supplies each one; the theory is parametric in all of them.

- (1) a set $\mathbb{I}$ of **identities**: abstract tokens with decidable equality and *no* internal structure;
- (2) a set $\mathbb{T}$ of **types** (tags / kinds);
- (3) key spaces AttrName , StyleName , PropName and a **value** universe Val , with admissible carriers $\text{AttrVal}, \text{StyleVal} \subseteq \text{Val}$;
- (4) a set Ev of **event types**;
- (5) a set $\mathbb{H}$ of **handler names** and a set $\mathbb{P}$ of **payloads**;
- (6) a set $\text{Phase} = \{\text{capture}, \text{bubble}\}$ of dispatch phases.

Finite maps

A **finite map** $m : K \rightarrow V$ is a finite partial function from a key space K to a value carrier V ; $\text{dom}(m)$ is its finite key set and $m[k]$ the value at $k \in \text{dom}(m)$. Maps are equal iff they have the same domain and agree pointwise on it. We write $\emptyset$ for the empty map, $\{k \mapsto v\}$ for the singleton, and $m \setminus k$ for m with key k removed (domain restricted to $\text{dom}(m) \setminus \{k\}$). Three operations on maps over a common K recur throughout.

Override. $m_1 \oplus m_2$ is the map with

$$\text{dom}(m_1 \oplus m_2) = \text{dom}(m_1) \cup \text{dom}(m_2), \quad (m_1 \oplus m_2)[k] = \begin{cases} m_2[k], & k \in \text{dom}(m_2), \\ m_1[k], & \text{otherwise.} \end{cases}$$

So m_2 wins on shared keys (“later wins”). $\oplus$ is associative with identity $\emptyset$, so the finite maps over a fixed K form a monoid under $\oplus$ – non-commutative in general.

Agreement. m_1 and m_2 are **in agreement**, written $m_1 \sim m_2$, iff they coincide wherever both are defined:

$$m_1 \sim m_2 \iff \forall k \in \text{dom}(m_1) \cap \text{dom}(m_2), m_1[k] = m_2[k].$$

Agreement is reflexive and symmetric but *not* transitive (a tolerance relation); disjoint maps agree vacuously. Override commutes on a pair exactly when it agrees: $m_1 \oplus m_2 = m_2 \oplus m_1 \iff m_1 \sim m_2$.

Difference. Fix a **value projection** π choosing which aspect of a value is compared (the value itself, a default, or an effective value-or-default), with $\perp$ for “absent” and $\perp \neq v$ for any

value v . The difference of m_a from m_b collects the added, removed, and changed keys:

$$\begin{aligned}\text{added}(m_a, m_b) &= \text{dom}(m_b) \setminus \text{dom}(m_a), \\ \text{removed}(m_a, m_b) &= \text{dom}(m_a) \setminus \text{dom}(m_b), \\ \text{changed}_\pi(m_a, m_b) &= \{ k \in \text{dom}(m_a) \cap \text{dom}(m_b) \mid \pi(m_a[k]) \neq \pi(m_b[k]) \}, \\ \text{diff}_\pi(m_a, m_b) &= \text{added} \cup \text{changed}_\pi \cup \text{removed}.\end{aligned}$$

diff_π is empty iff $m_a = m_b$ under π ; its changed part alone is empty iff m_a and m_b agree on their overlap under π .

Local data

Each facet of an element's local state is a finite map over the appropriate key space; its value carrier fixes *what* a key maps to.

- (7) an **attribute map** A : a finite map with $K(A) \subseteq \text{AttrName}$ and values in AttrVal – the declared, serialisable state;
- (8) a **style** S : a finite map with $K(S) \subseteq \text{StyleName}$ and values in StyleVal – a distinguished facet carrying its own value grammar (lengths, colours, ...) and the cascade defined below;
- (9) a **property map** ρ : a finite map with $K(\rho) \subseteq \text{PropName}$ and values in Val – the live, computed state, not necessarily serialisable.

Events bind handlers per type; ordering of handlers matters, so the carrier here is a *sequence*.

- (10) a **listener** $\lambda = (h, \varphi, fl) \in \mathbb{H} \times \text{Phase} \times \text{Flags}$: a handler name with its phase and an (extensible) flag record fl (e.g. $\{\text{once}, \text{passive}\}$; $\text{Flags} = \{\star\}$ collapses to “no flags”);
- (11) an **event binding** E : a finite map with $K(E) \subseteq \text{Ev}$ whose value carrier is a listener sequence, so $E[e]$ carries $\langle \lambda_1, \dots, \lambda_m \rangle$, to be invoked in that order.

Map equality is extensional, so two bindings coincide on e iff they carry the same listener sequence in the same order.

Elements

- (12) an **element** $\delta = (\tau, A, S, E, \rho)$: a type together with an attribute map, a style, an event binding, and a property map.

The **element domain** D is the set of all such records; write $\delta.\tau, \delta.A, \delta.S, \delta.E, \delta.\rho$ for its projections. An element carries *no* identity and *no* children: these are supplied by the tree. This separation is exactly what later lets an element keep its identity while its content δ or its position changes. The **state of an element** is precisely $\delta \in D$: definable (a fully specified record) and modifiable (replaced by some δ' under the operations of the next section).

The UI tree

- (13) a **UI tree** $T = (N, r, p, \chi, \ell)$, where $N \subseteq \mathbb{I}$ is a finite set of **nodes**, $r \in N$ the **root**, $p : N \setminus \{r\} \rightarrow N$ the **parent** map (total on non-roots), $\chi : N \rightarrow \mathbb{I}^*$ the **child-order** ($\chi(n)$ lists n 's children, in order), and $\ell : N \rightarrow D$ the **labelling** ($\ell(n)$ is node n 's element), subject to well-formedness below.

Write the **child relation** $n \triangleleft m : \iff m \neq r \wedge p(m) = n$, with $\triangleleft^+$ its transitive closure (*proper descendant*) and $\triangleleft^*$ its reflexive-transitive closure (*descendant-or-self*). The **ancestors** of n are $\{a \mid a \triangleleft^* n\}$.

T is **well-formed** iff:

- (WF1) **Unique parentage.** p is a total function on $N \setminus \{r\}$ and $r \notin \text{dom}(p)$. (Single parentage is automatic, p being a function; only r is parentless.)
- (WF2) **Parent/child coherence.** For all $n, m \in N$: $m \in \chi(n) \iff (m \neq r \wedge p(m) = n)$, and $\chi(n)$ has no repeated entries (so $\chi(n)$ is a repetition-free ordering of $p^{-1}(n)$).
- (WF3) **Acyclicity.** $\triangleleft^+$ is irreflexive: no node is its own proper ancestor.
- (WF4) **Universal containment.** Every node is a descendant of the root: $\forall n \in N. r \triangleleft^* n$.
- (WF5) **Closure of references.** $\chi(n) \subseteq N \setminus \{r\}$ for all n (children exist; the root is nobody's child).

Proposition (WF4 is forced). On a finite N , (WF1)–(WF3) already entail (WF4). *Proof sketch.* Iterating p from any node yields, by acyclicity, a strictly non-repeating chain, which terminates by finiteness; it can only terminate at the unique parentless node r ; hence $r \triangleleft^* n$. $\square$

We nonetheless *state* (WF4) because it is the defining requirement of the structure: *the root contains every other element.*

Proposition (T is a rooted ordered tree). Under (WF1)–(WF5), $(N, \triangleleft)$ is a rooted tree with root r , sibling order given by χ , and $r \triangleleft^* n$ for every node. (*Single parent + acyclic + finite + root-connected $\Rightarrow$ tree.*) $\square$

State

The **state space** is

$$\mathcal{T} = \{ T \mid T \text{ satisfies (WF1)–(WF5)} \}.$$

The **state of the tree** is the whole tuple $T \in \mathcal{T}$; the **state of an element** n is $\ell(n)$. State equality is tuple equality: same nodes, root, parentage, order, and labels.

Definability. Every $T \in \mathcal{T}$ is a completely specified finite object, and every projection of it (any node's type / attributes / style / binding / properties; any node's parent / children; the root; membership in N) is a total, computable query. There is *no* hidden state: T is the description.

Remark (identity abstraction). If identities are to be treated as abstract, quotient $\mathcal{T}$ by bijective renaming of $\mathbb{I}$; then two trees coincide iff isomorphic as labelled ordered trees. If identities are observable (external handles), do not quotient. Both regimes are supported.

Mutation

Modifiable is made precise as: a fixed set Op of **operations**, each a partial function on states with explicit preconditions, that is *sound* (preserves well-formedness) and *adequate* (reaches every well-formed target). The dynamics of state is the transition system these induce. Local edits are finite-map updates; in particular the override of a key reuses $\oplus$ on a singleton.

Working state

To model detached fragments faithfully – a node may leave the document yet stay alive and re-insertable, and a fragment may be built off-document before attachment – the *operational* state is a forest with one distinguished document root.

A **working state** $W = (N, r, p, \chi, \ell)$ is as in a UI tree *except* that $p : N \rightarrow N$ is partial ($n \notin \text{dom}(p)$ marks n a **fragment root**), retaining coherence (WF2) and acyclicity (WF3), with $r \in N$, $r \notin \text{dom}(p)$, and every node reaching a unique fragment root by iterating p . A node whose fragment root is r is **attached**; otherwise **detached**. Let $\mathcal{W}$ be the set of working states.

The **document** $\text{Doc}(W)$ is the restriction of W to the attached nodes $\{n \mid r \triangleleft^* n\}$. By construction $\text{Doc}(W) \in \mathcal{T}$ is a well-formed UI tree, so “ r contains every UI element” holds for the attached part at all times. A working state with no detached nodes is a single UI tree, recovering the previous section exactly.

Operations

Each operation is a partial function $\text{op} : \mathcal{W} \rightarrow \mathcal{W}$, written *guard* $\triangleright$ *effect*. Write $\text{desc}(c) = \{m \mid c \triangleleft^* m\}$, let $\text{ins}(\sigma, i, x)$ insert x at index i of σ , and $\text{del}(\sigma, i)$ delete index i .

Structural.

- $\text{create}(\delta) \triangleright \iota \in \mathbb{I} \setminus N; N += \{\iota\}; \chi(\iota) := \langle \rangle; \ell(\iota) := \delta$ (fresh detached singleton)
- $\text{insertChild}(n, i, c)$ guard: $n, c \in N, c \notin \text{dom}(p), 0 \leq i \leq |\chi(n)|, n \notin \{c\} \cup \text{desc}(c)$
 - $\triangleright p(c) := n; \chi(n) := \text{ins}(\chi(n), i, c)$
- $\text{detach}(c)$ guard: $c \in N, c \neq r, c \in \text{dom}(p)$
 - $\triangleright$ remove c from $\chi(p(c))$; $p := p \setminus c$ (c becomes a root)
- $\text{remove}(c)$ guard: $c \in N, c \neq r$
 - $\triangleright$ detach c if parented; $N -= \{c\} \cup \text{desc}(c)$ (destroys the subtree)
- $\text{move}(c, n, i) := \text{detach}(c); \text{insertChild}(n, i, c)$ guard also $n \notin \{c\} \cup \text{desc}(c)$.

Local (identity and position unchanged; only $\ell(n)$ is updated, on any $n \in N$; each restated as a finite-map update):

- $\text{setType}(n, \tau) \triangleright \ell(n).\tau := \tau$
- $\text{setAttr}(n, k, v) \triangleright \ell(n).A := \ell(n).A \oplus \{k \mapsto v\}$
- $\text{removeAttr}(n, k) \triangleright \ell(n).A := \ell(n).A \setminus k$
- $\text{setStyle}(n, k, v) \triangleright \ell(n).S := \ell(n).S \oplus \{k \mapsto v\}$
- $\text{removeStyle}(n, k) \triangleright \ell(n).S := \ell(n).S \setminus k$
- $\text{setProp}(n, k, v) \triangleright \ell(n).\rho := \ell(n).\rho \oplus \{k \mapsto v\}$
- $\text{removeProp}(n, k) \triangleright \ell(n).\rho := \ell(n).\rho \setminus k$
- $\text{addListener}(n, e, \lambda) \triangleright \ell(n).E[e] := \ell(n).E[e] \cdot \lambda$
- $\text{removeListener}(n, e, \lambda) \triangleright$ drop occurrences of λ from $\ell(n).E[e]$.

(`setType` is offered for generality; a framework with immutable types omits it. `set...` on a key with no removal guard is a no-op when the key is absent.)

Soundness, adequacy, dynamics

Theorem (soundness / closure). For every operation and every $W \in \mathcal{W}$ meeting its guard, $\text{op}(W) \in \mathcal{W}$; hence $\text{Doc}(\text{op}(W)) \in \mathcal{T}$. *Proof sketch.* Local operations leave (N, r, p, χ) untouched, so coherence and acyclicity hold trivially. Structural operations update p and χ together (coherence); acyclicity is preserved exactly because `insertChild` and `move` require $n \notin \{c\} \cup \text{desc}(c)$; `remove` and `detach` only delete edges or nodes. $\square$

Definition (mutation transition system). With Op as above,

$$W \xrightarrow{\text{op}} W' \iff \text{op's guard holds at } W \text{ and } W' = \text{op}(W).$$

$(\mathcal{W}, \rightarrow)$ is the **mutation transition system**; the behaviours from W_0 are $\text{Reach}(W_0) = \{ W \mid W_0 \rightarrow^* W \}$. This system is the formal content of “modifiable”: the state changes by, and only by, these transitions.

Theorem (adequacy / reachability). Fix a root identity. For any two working states W, W' with that root identity, $W \rightarrow^* W'$ (up to renaming of non-root identities). *Proof sketch.* From W , `remove` leaves until only r remains, then drive $\ell(r)$ to $\ell'(r)$ by local operations; from this canonical state, `create` and `insertChild` build any target structure and local operations set every label. Hence the algebra constructs and reshapes *arbitrary* finite UIs. $\square$

Remark (patches; agreement-restricted batching). Sequential composition makes $(\text{Op}^*, \cdot, \langle \rangle)$ a monoid acting on $\mathcal{W}$; a **patch** is an element of Op^* , applied as one transition. A step whose guard fails admits two fault policies – *skip* (total semantics) or *abort* (atomic semantics) – both definable over the same Op . For a bulk attribute update U (a finite map) onto a node’s A : the merge is conflict-free iff $A \sim U$, and $A \oplus U$ applies it with the update winning – the cascade operator doing double duty as the update operator.

Events

Events connect mutation to the outside world: an occurrence at a node deterministically yields a state transition by routing through the tree and folding the affected handlers, whose effects are themselves operations.

Definition (handler interpretation). $\mathbb{H}$ is a set of handler *names*; the framework supplies an interpretation

$$\llbracket \cdot \rrbracket : \mathbb{H} \rightarrow (\mathcal{W} \times \Pi \rightarrow \text{Op}^*),$$

assigning each name a partial map from (current state, payload) to a sequence of operations – its effect. Naming handlers and interpreting them separately avoids the size/circularity paradox that $\mathbb{H}$ would otherwise create by appearing inside the very states handlers act on.

Definition (occurrence). An **event occurrence** is a triple $\varepsilon = (e, t, \pi)$ with type $e \in \text{Ev}$, target $t \in N$, and payload $\pi \in \Pi$.

Definition (path). For an attached target t , the **path** is the unique simple root-to-target node sequence

$$\text{path}(t) = \langle a_0 = r, a_1, \dots, a_k = t \rangle, \quad a_j \text{ the child of } a_{j-1} \text{ toward } t,$$

well-defined since $\text{Doc}(W)$ is a tree. A detached target dispatches within its fragment (path taken from the fragment root) or is undefined; the framework chooses.

Definition (two-phase schedule). The order is *capture* $\rightarrow$ *target* $\rightarrow$ *bubble*. The **schedule** $L(\varepsilon, W)$ is the list of (node, handler) pairs from walking $\text{path}(t)$:

- **Capture:** for $j = 0, \dots, k-1$, the handlers of a_j whose listener phase is capture, in registration order;
- **Target:** at $a_k = t$, all of t 's handlers (either phase), in registration order;
- **Bubble:** for $j = k-1, \dots, 0$, the handlers of a_j whose phase is bubble, in registration order.

(A once listener is scheduled but emits a `removeListener` on itself; passive restricts what its effect may contain.)

Definition (dispatch). Dispatch folds the schedule's effects over the state:

$$\text{Dispatch}(W, \varepsilon) = \text{foldl } \text{step } W \ L(\varepsilon, W), \quad \text{step}(W', (n, h)) = \text{applyAll}(W', \llbracket h \rrbracket(W', \pi)),$$

where `applyAll` applies the returned operations left to right (under the fault policy above). This yields the **event transition** $W \xRightarrow{\varepsilon} \text{Dispatch}(W, \varepsilon)$.

Definition (full labelled transition system). The complete dynamics is

$$(\mathcal{W}, \text{Lab}, \Rightarrow), \quad \text{Lab} = \text{Op} \uplus \text{Occurrence},$$

the union of the mutation transitions and the event transitions. For fixed $\llbracket \cdot \rrbracket$ and a single occurrence, `Dispatch` is a function (deterministic dynamics); concurrency is modelled by interleaving in $\Rightarrow$.

Remark (control signals). To model *stopPropagation* / *preventDefault* / default actions, let a handler return a record $(\text{ops}, \text{stop}, \text{defaultPrevented})$ instead of a bare `Op*`; *stop* short-circuits the remaining schedule when *stop* is set, and a framework-defined default-action operation is appended after the fold unless *defaultPrevented*. The core above is the *stop* = `false`, no-default case.

The structural point: event routing is determined by *tree position* (propagation through ancestors), not by a flat global bus. That is what makes the model DOM-like rather than merely “callbacks on nodes”.

DOM-like systems

A system is **DOM-like** iff it instantiates the primitive universes and provides

$$(\mathcal{W}, \text{Op}, \llbracket \cdot \rrbracket, \text{Dispatch})$$

satisfying (D1)–(D7).

- (D1) **Hierarchy.** Every state's document is a finite rooted ordered tree: one root, a unique parent per non-root, totally ordered siblings. (*WF1–WF3.*)
- (D2) **Universal containment.** The root is an ancestor of every attached element; the UI is exactly the set of nodes the root contains. (*WF4; Doc.*)
- (D3) **Stable identity.** Elements are named by identities invariant under change of content or position: an operation may alter $\ell(n)$, $p(n)$, or n 's index, yet n remains the same element. Identity $\neq$ content $\neq$ position. This licenses modify-in-place and move.
- (D4) **Stateful elements.** Each element carries $\ell(n) = (\tau, A, S, E, \rho)$ – type, attribute map, style, event binding, property map, each a finite map (bar τ). The element's state is fully given by $\ell(n)$; the tree's by W .
- (D5) **Closed, adequate mutation algebra.** There is an operation set Op of guarded partial functions on states that is sound (preserves well-formedness) and adequate (generates every finite well-formed tree); the dynamics is $(\mathcal{W}, \rightarrow_{\text{Op}})$.
- (D6) **Event-driven, tree-directed dynamics.** Elements bind handlers to event types; an occurrence at a target induces a transition by dispatching along the root-to-target path in two phases (capture / bubble), folding handler effects – themselves operations – over the state.
- (D7) **Full observability.** The entire state is inspectable by total queries (per node: type, attributes, style, binding, properties, parent, children; per tree: root, membership). Query and mutation are the only interactions; no hidden state.

Definition. A system satisfying (D1)–(D7) is **DOM-like**. The facets A, S, E, ρ and the operation set above constitute the *canonical* DOM-like system; any instantiation of the universes and any Op meeting (D5) qualifies. Minimally: rooted-ordered-identity tree (D1–D3) + stateful nodes (D4) + sound-and-adequate edit algebra (D5) + tree-directed dispatch (D6) + transparency (D7).

Equivalent presentations and extensions

Recursive presentation. For a single-rooted tree, the relational form (N, r, p, χ, ℓ) is isomorphic to the least solution of

$$\text{Tree} \cong \mathbb{I} \times D \times \text{Tree}^* \quad (\text{a node} = \text{identity} \times \text{label} \times \text{ordered list of child trees}),$$

the initial algebra of $\text{node} : \mathbb{I} \times D \times \text{List} \rightarrow \text{Tree}$. The relational/identity form makes mutation and identity-sharing first-class (preferred operationally); the nested form makes structural induction and persistent rebuilds natural (preferred for proofs).

Style as a distinguished attribute. A style is isomorphic to a single attribute whose value is itself a finite map; a framework wanting “everything is an attribute” folds S (and ρ) into A over a richer carrier, leaving (WF1)–(WF5) untouched.

Cascade $= \oplus$. The effective style of a node under an ordered list of contributing styles $S_{\text{base}}, S_1, \dots, S_m$ (e.g. inherited, class, inline) is the left fold

$$S_{\text{base}} \oplus S_1 \oplus \dots \oplus S_m$$

in the monoid (finite maps, $\oplus, \emptyset$) defined above – “later wins”, so the inline contribution is placed last. The same fold defines attribute resolution across precedence layers.

Reconciliation $= \text{diff}$. Between consecutive element states, the work a renderer must perform is, facet by facet, diff_π on the corresponding finite maps: added, changed $_\pi$, and removed keys give exactly the attribute / style / property edits to apply, with the projection π selecting declared value, default, or effective value-or-default. To skip selected keys, filter the result by an exemption predicate ε . Thus patch generation reuses the difference of the preliminaries, not new code.

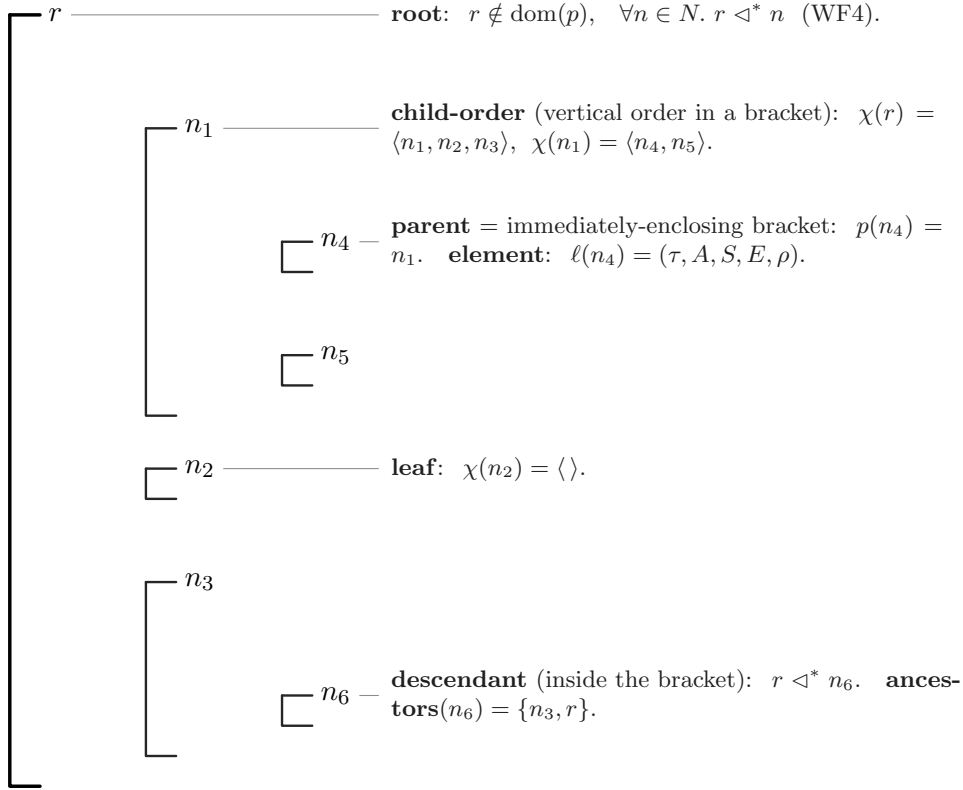
Typed / schema-constrained DOM. Refine Val into a typed value algebra and a schema $\Sigma : T \rightarrow (\text{admissible facet signatures})$; add **(WF6)** requiring $\ell(n)$ to conform to $\Sigma(\ell(n).\tau)$, and restrict each operation’s guard to preserve it. The underlying theory is unchanged.

Extensibility. D is a product of finite maps, so new local facets (namespaces, reconciliation keys, shadow-subtree pointers, derived caches) are added as further map components without disturbing the tree axioms; events extend via Flags, Phase, and control signals; batching via the patch monoid.

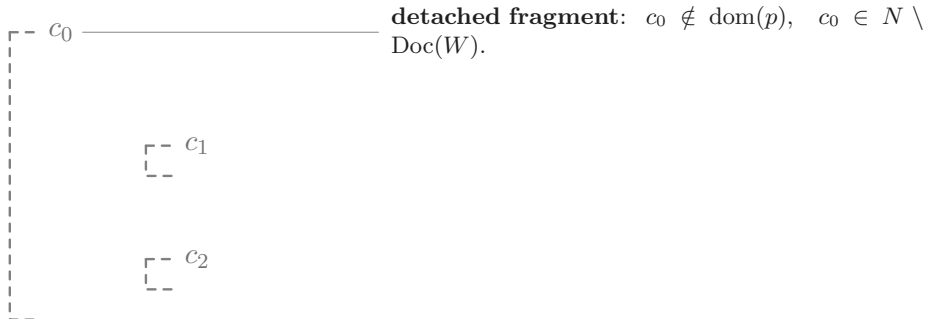
Index of the requested notions. UI tree – Def. (13), well-formedness (WF1)–(WF5), rooted-tree proposition; element – Def. (12); attributes – Def. (7); style – Def. (8) and cascade $= \oplus$; events – Defs. (10), (11) and dispatch; state – state space $\mathcal{T}$; modifiability – the operation algebra with soundness and adequacy; DOM-like – (D1)–(D7).

The UI tree $T = (N, r, p, \chi, \ell)$ as nested brackets

Reading: a left bracket $[$ encloses a node with its whole subtree — nesting = ancestry, indent = depth, vertical order = child-order χ . Each $n_i \in \mathbb{I}$ is an identity; $\ell(n_i) = \delta = (\tau, A, S, E, \rho)$ is its element.

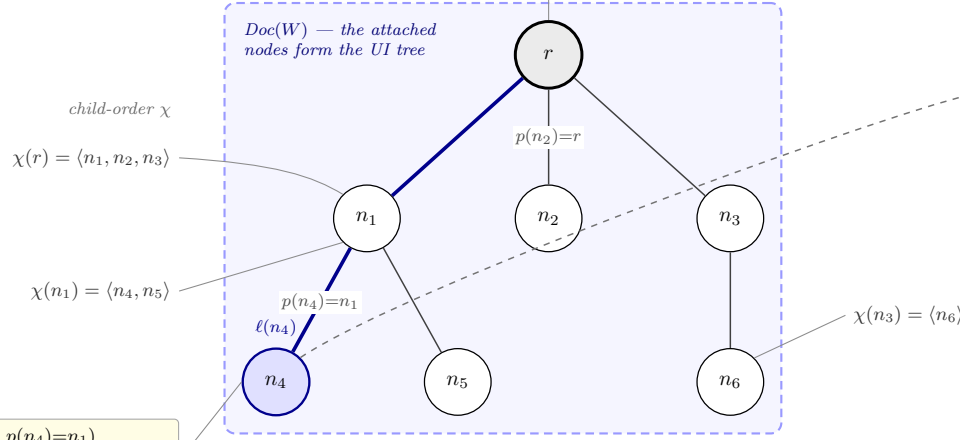


a second top-level bracket — detached



A UI tree $T = (N, r, p, \chi, \ell)$ **with a detached fragment** (a working state W)

root r : $r \in N$, $r \notin \text{dom}(p)$; $\forall n \in N. r \triangleleft^* n$ (WF4: the root contains every node)



$\ell(n_4) = \delta = (\tau, A, S, E, \rho)$ — element record

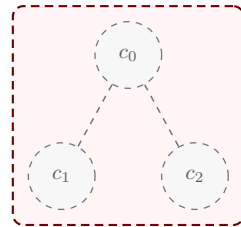
$\tau = \text{button}$	type τ
$A = \{ \text{id} \mapsto \text{"ok"}, \text{disabled} \mapsto \text{false} \}$	attribute map A
$S = \{ \text{color} \mapsto \text{blue}, \text{padding} \mapsto 4 \}$	style S
$E = \{ \text{click} \mapsto \langle \lambda_1, \lambda_2 \rangle \}$	event binding E
$\rho = \{ \text{focused} \mapsto \text{true} \}$	property map ρ

each facet is a finite map ($\oplus$ overrides, $m \setminus k$ removes)

Legend

$N \subseteq \mathbb{I}$	nodes (identities); attached $\cup$ detached
r	root — the unique node with no parent
p	parent map: $p(m) = n \Rightarrow n$ is the parent of m
χ	child-order: $\chi(n) =$ ordered children of n
ℓ	labelling: $\ell(n) = \delta$, the element of node n
$\delta = (\tau, A, S, E, \rho)$	type, attribute map, style, event binding, property map
$n \triangleleft m$	m is a child of n ($p(m) = n$)
$n \triangleleft^* m$	m is a descendant of n (reflexive–transitive)
$\text{Doc}(W)$	the attached part — a well-formed UI tree
$\perp$	no value / no parent

detached fragment



$c_0 \notin \text{dom}(p)$ — a fragment root.
In N , but *not* in $\text{Doc}(W)$.
Re-attach with $\text{insertChild}(n, i, c_0)$;
then c_0 becomes attached.